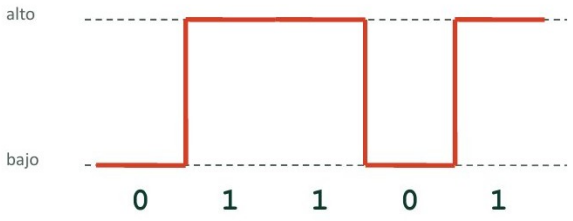
Ivett Anahi Zaragocin Mora

Bits y E/S mapeada en memoria

SESIÓN 2 · SISTEMAS DIGITALES

Un bit es un nivel de voltaje



Una señal digital: 0 = nivel bajo, 1 = nivel alto

Nivel	Se escribe	Arduino Uno	ESP32
Bajo	0	≈ 0 V	≈ 0 V
Alto	1	≈ 5 V	≈ 3,3 V

Es el **transistor como interruptor**: en **corte** da 0 y en **saturación** da 1.

Un byte: ocho bits, cada uno con su peso

Regla Bits numerados de **derecha a izquierda** (bit 0 a la derecha). Valor del byte = **suma de los pesos** de los bits en 1; peso del bit n = 2^n.

128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	1
7	6	5	4	3	2	1	0

= 4 + 1 = 5

bits 2 y 0 en 1

128	64	32	16	8	4	2	1
1	0	0	0	0	0	0	1
7	6	5	4	3	2	1	0

= 129 = 0x81

0x81 (apunte a mano)

Hexadecimal: 4 bits en un símbolo

1 dígito hex = **4 bits** (*nibble*); 1 byte = 2 dígitos hex.

0010 0000 = 0x20

» `0x` = hexadecimal; `0b` = binario (`0b00100000`).

0000 **0**	0001 **1**	0010 **2**	0011 **3**
0100 **4**	0101 **5**	0110 **6**	0111 **7**
1000 **8**	1001 **9**	1010 **A**	1011 **B**
1100 **C**	1101 **D**	1110 **E**	1111 **F**

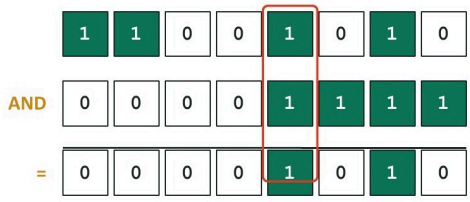
Tabla binario ↔ hexadecimal (0 a F)

Compuertas lógicas, un bit a la vez

A	B	AND &	OR |	XOR ^	NOT A ~
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

AND 1 si **ambos** son 1; apaga (con 0) y pregunta
OR 1 si **alguno** es 1; enciende (con 1)
XOR 1 si son **distintos**; invierte (con 1)
NOT **invierte** el bit

Bit a bit: columna por columna



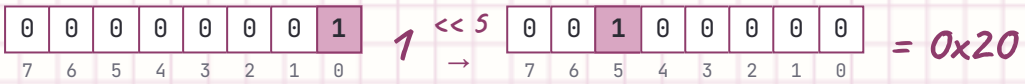
0xCA & 0x0F: cada columna es un AND independiente

- Cada columna es un AND independiente.
- La CPU hace las **8 a la vez**.

```
resultado = 0xCA & 0x0F; // 0x0A
```

La máscara: 1 << n señala un bit

Máscara byte con 1 solo en los bits a tocar. `1 << 5` = «el bit 5» = `0x20`.



Las cuatro recetas

Quiero	En C	Idea
Poner el bit *n* en **1**	`REG |= (1<<n);`	OR con 1 da 1
Poner el bit *n* en **0**	`REG &= ~(1<<n);`	AND con 0 da 0
Invertir el bit *n*	`REG ^= (1<<n);`	XOR con 1 invierte
Preguntar por el bit *n*	`if (REG & (1<<n))`	≠ 0 solo si el bit es 1

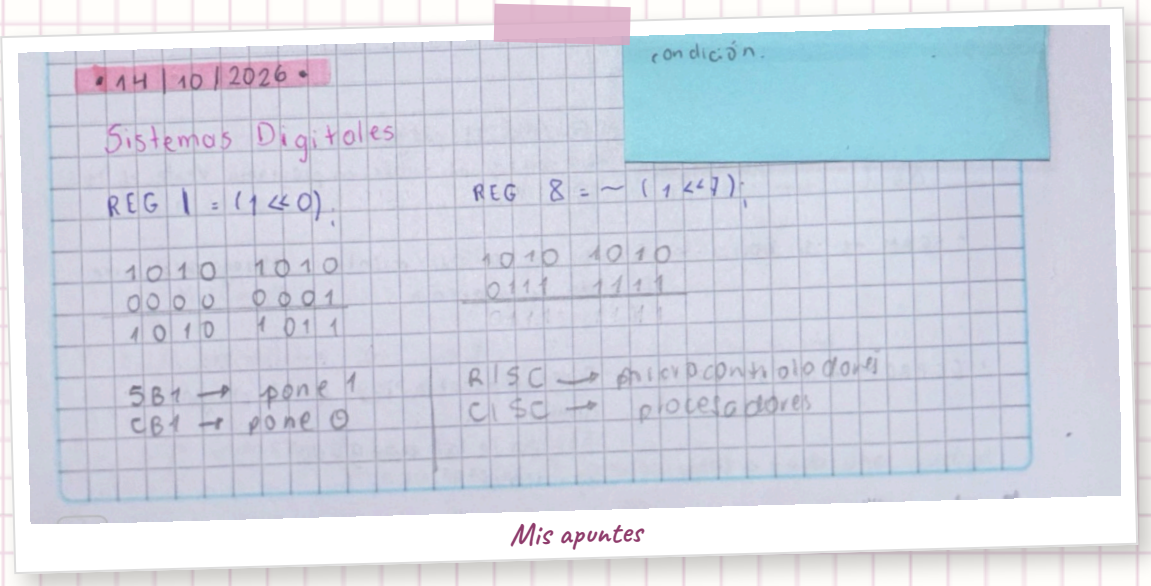
El juego de instrucciones del AVR

Instrucción	Qué hace
`LDI r16, 0x20`	Carga el número 0x20 en el registro r16
`ADD r16, r17`	Suma r17 a r16
`AND OR EOR COM`	AND, OR, XOR y NOT bit a bit: las instrucciones lógicas
`SBI 0x05, 5`	Pone en **1** el bit 5 del registro de E/S 0x05 (PORTB)
`CBI 0x05, 5`	Pone en **0** ese mismo bit
`RJMP`	Salta a otra parte del programa

Ejercicios

REG = | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | = 0xAA, base de cada ejercicio.

#	Ejercicio	Resultado
1	`REG |= (1<<0);`	`1010 1011` = 0xAB
2	`REG &= ~(1<<7);`	`0010 1010` = 0x2A
3	`REG ^= 0x0F;`	`1010 0101` = 0xA5
4	¿Se cumple `if (REG & (1<<3))`?	**Sí**: AND = 0x08 ≠ 0
5	Enciende los bits 0 y 1 sin tocar los demás	`REG |= (1<<0) | (1<<1);`
6	¿Qué falla en `REG = (1<<2);`?	`=` **sobrescribe** el registro; falta `|=`.



Mis apuntes

RISC y CISC; 8 y 32 bits

- **RISC**: pocas instrucciones simples, casi todas en 1 ciclo (AVR, ARM, RISC-V, Xtensa). **CISC**: muchas, algunas elaboradas (x86).
- **8 o 32 bits** = bits que procesa la CPU de una vez. Sumar 32 bits: ATmega328P (8 bits) necesita **4 sumas**; ESP32, **una**.

Demostración: de C a bits

COMPILACIÓN

Lenguaje C	→	`PORTB |= (1 << 5);`
Ensamblador	→	`sbi 0x05, 5`
Código máquina en la Flash	→	`1001 1010 0010 1101 (0x9A2D)`

1 línea de C = **1 instrucción de 2 bytes**. SBI: `1001 1010` opcode · `0010 1` = E/S 0x05 (memoria **0x25**) · `101` = bit 5.

Funciones de Arduino y registros

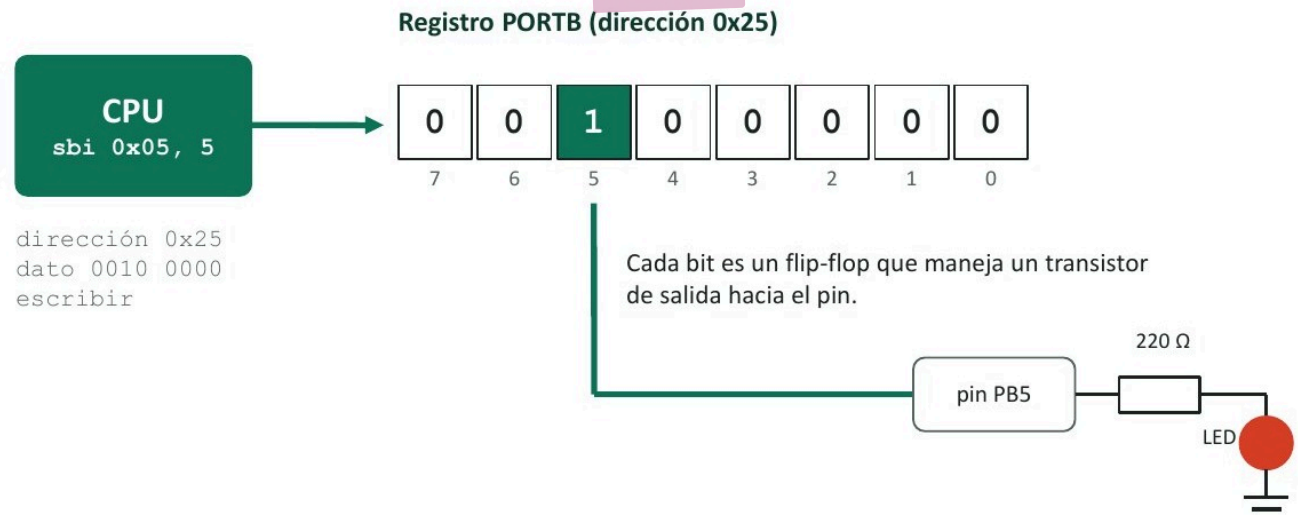
Arduino

```
pinMode(13, OUTPUT);
digitalWrite(13, HIGH);
digitalWrite(13, LOW);
pinMode(2, INPUT_PULLUP);
int b = digitalRead(2);
```

Registros

```
DDRB  |=  (1 << 5);
PORTB |=  (1 << 5);
PORTB &= ~(1 << 5);
DDRD  &= ~(1 << 2);
PORTD |=  (1 << 2);
int b = (PIND & (1 << 2)) != 0;
```

Escribir un bit enciende un LED

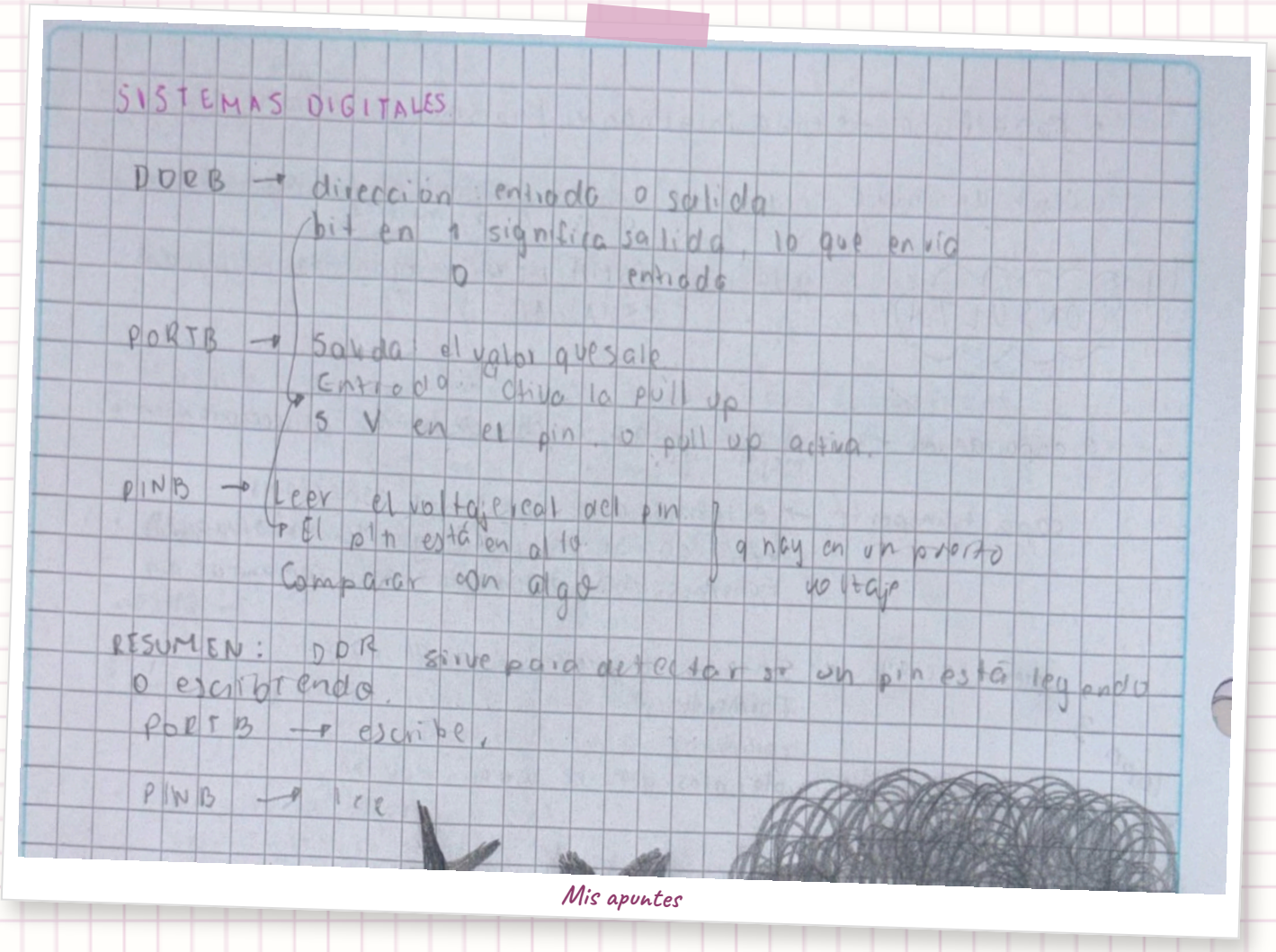


E/S mapeada en memoria: PORTB tiene dirección (0x25) como una celda de memoria.

Tres registros por puerto

Registro	Para qué sirve	Bit en 1 significa
DDRB	Dirección: entrada o salida	Salida
PORTB	Salida: valor que sale. Entrada: activa pull-up	5 V, o pull-up activa
PINB	Leer el voltaje real del pin	El pin está en alto

Bit *n* del registro = pin *n* del puerto. ATmega328P: puertos B, C y D.



Mis apuntes

Los cuatro estados de un pin

DDRxn	PORTxn	Estado del pin	Ejemplo
0	0	Entrada **flotante** (alta impedancia): lee al azar	Nada conectado
0	1	Entrada con **pull-up**	Botón a GND (suelto 1, pulsado 0)
1	0	Salida en 0 (0 V)	LED apagado
1	1	Salida en 1 (5 V)	LED encendido

Resumen rápido

1. Bit = nivel de voltaje; byte = 8 bits; dígito hex = 4 bits.
2. Máscara `1 << n`: `|=` pone en 1, `&= ~` en 0, `^=` invierte, `&` pregunta.
3. Los periféricos tienen dirección, como la memoria.
4. DDR elige entrada/salida, PORT escribe, PIN lee.